```
function makeAjaxCall(id) {
  return $.ajax({
    url: `projects/${id}.js`
    dataType: "json"
  })
}

function loadFromData(data) {
  const project = new Project()
  project.name = data.project.name
  // and so on
}

function loadProject(id) {
  return makeAjaxCall(id).then((data) => this.loadFromData(data))
}
```

This isn't your actual passing code; that code will be inside a class.

With the structure cleared away behind separate methods with intention-revealing names, two strategies become visible:

- If the callback is defined as a separate function or method, you can test it directly with canned data.
- If the callback is separate, you can stub the Ajax call and make sure it triggers the loader.
- The combination of the two can still be handled by the Capybara integration test.

Here are some Jasmine tests. The first takes data and passes it directly to a method to load the data; the second uses the testdouble.js library to stub the Ajax call:

js_jasmine/02/spec/javascripts/project_load_spec.js

```
import {Project} from "../../app/javascript/packs/project.js"
import {ProjectLoader} from "../../app/javascript/packs/project_loader.js"
import td from "testdouble/dist/testdouble"
import tdJasmine from "testdouble-jasmine"
tdJasmine.use(td)

describe("Project Loader", () => {
  let input

  beforeEach(() => {
    input = {project: {name: "Project Runway",
      tasks: [{title: "Start Project", size: 1},
        {title: "End Project", size: 1}]}}
  })
```